KIN.

LIVING WORLDS / CROSSING SCREENS

Product Vision, Creature Autonomy, Cross-Device Magic, and Engineering Blueprint

Build packet | 22 October 2026 | Source code contracts and agent prompts included in companion ZIP

> “A KIN is not a pet trapped in an app. It is a little life that can cross screens, make friends, compete, and come home changed.”

PRODUCT VISION

KIN — Living Worlds, Crossing Screens

Master Product Vision & Autonomous Coding-Agent Build Packet

Version: 1.0 | **Date:** October 22, 2026 | **Stage:** Build blueprint, not shipping specification

> **THE MOMENT:** Place two phones side by side. Your tiny creature notices another world, pads toward the seam, glances back at you, hops off your screen and lands on your friend's screen. Two more devices join; their creatures gather, play, compete and remember each other. The experience is delightful whether the devices are iPhone, Android, tablet, foldable or web.

North star: A KIN isn't a pet trapped in an app. It is a persistent, portable, autonomous digital being whose identity, memories, skills, relationships and appearance survive changes of screen, device, world and network.

To the coding agent: Do not build a conventional dashboard or generic AI chat app. Deliver a magical, immediately playable *two-device crossing demo* first. Use small high-quality worlds, a beautiful creature and robust deterministic state. Do not build an MMO, a token economy, or open-ended LLM simulation before the crossing is convincing. Every feature must serve **bond** → **express** → **gather** → **compete** → **remember**.

Product principles

1. **Three seconds to delight.** A live creature visible immediately; no signup wall.

2. **One meaningful input, one expressive response.** Pet, toss, invite, cross, cheer.

3. **Crossing is ceremony, not a loading screen.** Glance → intent → invite → edge alignment → synchronized movement → arrival → social reaction.

4. **Autonomy is observable.** A KIN independently chooses actions based on needs, personality, memories, context and safety constraints.

5. **Offline is real.** Solo works with radios off; camera-only encounters are possible where geometry allows; real-time nearby play needs a working optical/local wireless/LAN transport.

6. **One universe, multiple platforms.** iOS, Android and web share protocol, simulation, assets and account-independent local identity.

7. **Make social play opt-in.** Nearby discovery, visits, camera, name exposure and invitations require clear controls and consent.

8. **Monetize expression, never advantage.** No pay-to-win stats or manipulative care/death timers.

9. **Performance is a feature.** Crisp typography, restrained color, exceptionally smooth motion, modest power use.

10. **No proprietary IP.** Original creature silhouettes, worlds, cosmetics and naming; don't reuse Pokémon/Fortnite/Minecraft game assets without actual grants.

Experience thesis

The app answers five questions a player feels rather than reads: **Who is my KIN? What can it do? Can it visit yours? Will it win? What did it remember?**

The emotional product loop

Hatch → name → bond → care/play → learn → dress → cross to a gathering → surprise social behavior → party/sport/battle → return home with memory/earned expression → repeat.

Audience and positioning

Families and kids with parental controls, teens who enjoy customization/collection, adults who want a cozy companion, friends who compete casually, streamers and event attendees. All onboarding and public social features must be age-appropriate. Do not assume users are children or that the service is safe for minors without compliance work.

The flagship: KIN CROSS

Trigger: two devices placed edge-to-edge, each already in a shared encounter or invitation flow.

Ten-beat directed sequence:

1. Each local KIN detects a nearby invitation; player can allow, decline or restrict visits.

2. The games exchange *minimal* visitor state and agree on an encounter ID.

3. Each screen presents an edge guide. Players align devices roughly; no precise device-location tracking needed in MVP.

4. KIN turns its head toward the seam, notices the other character, chirps, and takes two careful steps.

5. Destination KIN reacts independently: eager, cautious, playful, aloof, based on its temperament and memories.

6. Source sends `cross.prepare` and destination confirms compatible assets, available capacity and ready-to-spawn.

7. Both agree on an absolute/relative presentation start using measured clock-offset approximation or acknowledgement barrier; a shared animation script takes over.

8. Source character exits past the chosen edge and disappears; destination spawns same unique KIN ID at opposing edge and continues arrival movement, shadow, squash/stretch and sound.

9. Destination acknowledges `cross.commit`; ownership remains with source, visit lease is time-bound. Source shows a postcard/return portal, never a duplicated independent pet.

10. On return/disconnect, creature goes home with a durable encounter memory; reconcile any local events when possible.

Engineering truth: These are *two local renders coordinated by messages*, not a sprite literally transported across glass. The illusion must be impeccable. Physical alignment can be manual initially. When devices are not physically adjacent, use a portal animation instead of pretending edges align.

Failure paths: source walks away, destination closes app, unexpected packet loss, capacity reached, permission denied, mismatched asset version, host migration. Never lose or duplicate a KIN; show graceful `cross.cancel`/`cross.return`, use an authoritative lease or owner as source of truth.

Wow acceptance: Two dissimilar phones on the same LAN show one visually continuous exit/entry with correct identity and skin, arrival reaction and return home. In 10 consecutive demonstrations there is no orphaned or permanently duplicated creature. Exact latency/performance targets must be measured on real devices.

The pillars

KIN HOME — the private, comforting space

An immediate micro-habitat (desk-sized diorama), petting response, sleep, play, idle routines, memories and decor. Initial world may be one beautiful prebuilt GLB from original/appropriately licensed assets. No infinite map. Keep personalized interactions meaningful without anxiety-inducing decay.

KIN PARTY — gatherings and spontaneous group play

2–8 device initial target; later 10–30 or more with coordinator and interest management. Visitors arrive through individual edge/portal animations. Creatures can cluster, discover toys, dance, compete in spontaneous mini-games, share benign gestures, pose for group photos and form friendships or rivalries. Let users spectate or leave freely. Provide party roles, consent, capacity and host handoff.

KIN ARENA / BATTLE — expressive, fair contests

Short, skill-influenced competitions: dodgeball, tug-of-orb, timing duels, obstacle navigation, capture-the-light. Avoid violence and RPG complexity as default. Owner trains abilities and picks a strategy; KIN autonomously executes bounded decisions. Server or local host owns deterministic scoring. Transparent outcomes; cosmetics do not affect power.

KIN SPORTS — deep replayability

First sport: **KIN DASH**, a 20–45-second obstacle race; input: pre-race stance/strategy and one limited cheer/boost timing mechanic if desired. Train agility, composure, sprint and obstacle reading through simple mini-games. Future: KIN Hoops (2v2 arcade shot selection), Relay, Glider, Frisbee, Dance-Off, Parkour, trick trials. Equip jerseys, shoes, celebrations, team colors, flags and stadium themes. Add asynchronous ghosts, daily tracks and leagues. Training progression must be capped and understandable.

KIN SOCIAL — friendships with memory

Consent-based meets, recurring recognition, friend/rival affinity, visits and photo memories. Creature-to-creature interactions aren't unrestricted AI conversations: structured intents such as `greet`, `invite`, `offer_toy`, `accept`, `decline`, `cheer`, `challenge`, `comfort`, `leave`, `trade_cosmetic_preview`. Avoid transmitting personal conversations or sensitive behavioral profiles. Shared memories require bilateral agreement or local-only storage.

KIN STYLE / SKINS — visible individuality

Species archetype + body palette + surface material + ears/tail/horns + eye shapes + accessories + outfits + sports uniform + emote pack + walk cycle + victory celebration + habitat theme. Visible in arrival sequence and group photos. Layer cosmetics atop the same animation skeleton where possible; enforce silhouette and performance budgets. Keep acquisition and rarity claims transparent. No randomized paid loot for initial release.

KIN AVATAR — owner's presence

Optional lightweight human-facing profile, badges, nickname, friend circle, style, stadium/team branding and accessibility preferences. Do **not** require an AI face scan or real name; support pseudonyms and private profiles. Avatar representation may start as a 2D card, progressing to stylized 3D later.

KIN CLUB / CAMP / PARADE

Clubs coordinate private recurring events and sports teams. Camp is slower cooperative habitat play. Parade is cosmetic shows, judging, group photos and seasonal themes. These are later modes and should reuse the same encounter/session infrastructure.

What makes the creature feel alive

Each KIN has immutable creature ID, species, genetic seed, traits, bounded drives (energy, curiosity, comfort, sociality), relationship memories, skill progression, current mood and an active goal. The behavior system observes world events, scores goals, selects a short plan and executes animation/action through a sandboxed ability vocabulary.

Recommended decision stack: deterministic utility AI / behavior tree (fast frequent decisions) + episodic structured memory + optional AI director for occasional surprise, novel safe micro-behaviors and reflection. No cloud inference every frame. No unrestricted agent tool calls. The brain must be testable without graphics or network.

Example: Lumi is curious (0.9), socially brave (0.6), low energy (0.3), and remembers Nox as a friendly competitor. At a party it may approach Nox, wave, invite a low-energy toy game, then choose to rest. Nox independently may accept or decline. Unscripted *selection* atop scripted/safe *actions* feels autonomous.

Memory layers: working memory (seconds), session episodic log (party actions), relationship summary (bounded), long-term stable preferences (carefully earned), player-visible scrapbook. Avoid falsely claiming the pet is conscious. Let owners reset/retrain personality influences and delete memories.

Airplane / no-network promise

Tier 0: radio silence, zero network. Fully playable local creature, offline bundled maps/content, local mini-games, local saves. Optical QR-like introductions and turn-taking when one camera can see another screen; camera permission and positioning required. No invisible synchronized multiplayer without some physical channel.

Tier 1: local network no internet. Cross-device party via shared Wi-Fi LAN (including a travel router if allowed). Multiplatform web/native sessions with local host; discovery may require manual room code/QR and LAN-reachable server.

Tier 2: permitted nearby radios, no internet. Optional native peer transports on supported devices; feature-detect, never assume cross-platform universal support. Airplane policies and OS toggles apply.

Tier 3: online. Remote events, cloud backup, friends, clubs, matchmaking and global tournaments.

Offline web limitations: first installation and content pre-caching are required, HTTPS/PWA support varies, tab suspension can stop execution, camera permissions apply. Background life resumes from persisted timestamps and bounded elapsed-time simulation; not continuous execution in a suspended app.

Design direction — Tesla restraint, creature warmth

Graphite #0B0D12; porcelain #F6F6F3; soft mint #6EE5CA; periwinkle #9EA8FF; sunset amber #E9AF6B. The world is softly tactile, 3D characters richly expressive; UI appears only when needed. Minimal HUD, thin labels, legible type, no generic neon gamer clutter. On-screen touch zones clear. The first screen is the creature itself. Cinematic but fast edge transfers, tasteful squash/stretch, eye-tracking, anticipation, overshoot and landing dust. Ambient audio subtle and instantly muteable. Haptics progressive enhancement only.

Primary screens

1. **Home:** creature centered in miniature habitat, 0–3 contextual affordances, bottom icon rail appears after touch.
2. **Gather:** discover/scan/join + visible privacy controls, detected peers and party status.
3. **Crossing:** paired edge guides, readiness, synchronous departure/arrival animation.
4. **Party:** one scene with many creatures and emergent gestures; owner may highlight their KIN.
5. **Arena:** matchmaking lobby, lane lineup, short sport, outcome replay.
6. **Wardrobe:** 3D orbit model, materials/outfits presets and outfit saves.
7. **Memory Book:** visual encounters with date, other KIN alias (consented), pictures and notable events.
8. **Profile/Club:** avatar, badges, teams, safety/settings and friends.

Accessibility

Reduced motion (replace hopping with fades), color-blind-safe UI, captions for chirps/sound cues, screen reader labeling of game controls, alternate touch inputs for all motion/fold mechanics, haptics/audio toggles, scalable text outside 3D, avoidance of flashing patterns.

Product constraints and risk register

- Camera-only **two-way real-time** interactions cannot be promised for arbitrary physical arrangements; use optical introductions or single-device guest display, and native/LAN transport when present.
- Do not advertise Wi-Fi Aware as universal Android↔iPhone↔browser; platform capability and interoperability testing required.
- iPhone foldable device APIs are optional capability extensions, not the core specification; never make basic gameplay hinge-dependent.
- Never trust rendered visuals or camera-read codes for verified ownership or rare-item transfers.
- Multiplayer simulation must have one authoritative outcome per event and disconnect recovery.
- AI-generated content must not transmit user private data to other KIN or services without consent.
- Children require protective defaults, restricted discovery/communication and platform/region-specific legal review.
- Avoid clones of existing mascot characters, protected map assets, and proprietary game universes.
- GLB/world asset licenses must be inventoried and credited where required.

Implementation map

Detailed architecture: `architecture/SYSTEM_DESIGN.md`.

Types and event protocol: `contracts/kin-protocol.schema.json`, `contracts/kin-types.ts`.

Coding-agent operating instructions: `agent/AGENTS.md`, `agent/BUILD_PROMPT.md`.

Phased backlog and acceptance: `delivery/ROADMAP.md`.

Art direction and screen walkthrough: `design/CREATIVE_DIRECTION.md`.

Verified reference documentation

- Babylon.js engine capabilities: https://www.babylonjs.com/specifications/
- Capacitor cross-platform runtime: https://capacitorjs.com/docs
- Apple Wi-Fi Aware capabilities and device support: https://developer.apple.com/documentation/WiFiAware
- Apple iOS Wi-Fi developer note: https://developer.apple.com/documentation/technotes/tn3111-ios-wifi-api-overview
- Browser camera permissions and secure contexts: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia
- Kenney open game art: https://kenney.nl/assets (verify per-pack licenses)

SYSTEM ARCHITECTURE

KIN System Architecture — Implementation Blueprint

Decision record

Primary: TypeScript monorepo; Babylon.js rendering with WebGL baseline and WebGPU opportunistic; React (or lightweight UI) for menus; Capacitor native containers for iOS/Android; browser PWA; shared behavior/simulation engine without DOM dependencies. Platform-agnostic persistence and transport interfaces. Initially ship a local network crossing. Do not assume Capacitor gives cross-platform wireless peer discovery out of the box.

Modules

`apps/web`	Browser/PWA + offline shell
`apps/mobile`	Capacitor shell + native plugins
`apps/dev-lab`	Simulated multi-peer QA, performance harness
`packages/core`	Identity, creature state, drives, needs, skills, memory, utility AI
`packages/sim`	Fixed-step world simulation, deterministic rules, event reducer
`packages/renderer`	Babylon scenes, animation state graph, GLB loading, LOD
`packages/interaction`	Gesture/touch/accessibility, care activities
`packages/protocol`	Schemas, canonical encoding, validation, version migration
`packages/transports`	In-process, LAN WebSocket, optical, relay, native adapters
`packages/session`	Discovery, invites, leader election, leases, cross transaction
`packages/persistence`	IndexedDB and native database adapter, migrations, event log
`packages/content`	Versioned species, cosmetics, habitats, activities, assets
`packages/ai`	Intent director interface; local deterministic provider initially
`services/relay`	Optional authenticated real-time signaling/relay
`services/sync`	Optional user account, data backup and cross-device sync
`services/content`	Manifest/CDN/seasonal content, optional remote config

Core hard boundaries

- Renderer is read-only consumer of simulation snapshots and emits *intent* events; no game rules in Babylon components.
- Creature brain selects actions, world rules decide whether they are valid, renderer decides how to depict them.
- Session coordinator resolves shared state; ownership and creature profile updates require explicit authority rules.
- Every transport sends the same versioned protocol envelope; optical supports subset and reduced payload size.
- Local persistence stores owner profile, creature snapshot, append-only significant events, outstanding visits and sync cursor.

State authority model

Home owner: authoritative immutable identity, permanent cosmetics, long-term progression and personal memories.

Session host: authoritative temporary positions, collisions, race outcome, event order and session clock. Clients submit bounded intents. Host cannot permanently transfer ownership by claiming visitor events.

Cloud sync (if enabled): authenticates ownership and durable shared awards, resolves versions, prevents duplicate rewards. Offline activities are provisional until verified where competitive/economic value exists.

Crossing transaction

States: `AT_HOME` → `INVITED` → `PREPARING` → `READY` → `EXITING` → `VISITING` → `RETURNING` → `AT_HOME`; cancellation from any pre-commit state and timeout/lease recovery from visiting.

Protocol steps: `visit.invite` → `visit.accept` → `cross.prepare` → `cross.ready` → `cross.start` → `cross.commit` → `visit.arrived` → `visit.heartbeat...` → `visit.return` → `visit.complete`.

Invariant: there is only one **authoritative active home/visit lease** per KIN. Display projection may temporarily include both source exit sprite and destination entry sprite during a choreographed handoff; never persist two owned creatures.

Sync: use logical timestamps and monotonically increasing sequence per session, send animation start sufficiently in the future relative to estimated clock offset, send idempotent acknowledgments. Falling behind should skip to a visually reasonable phase rather than blocking entire party. For first demo same-LAN RTT can be measured; no impossible exact frame lock promise.

If visit interrupted, owner retains KIN ownership; destination eventually expires lease and removes visitor; after reconnection reconcile events by event ID. On UI restart the home view displays safely with `traveling/returning` indicator or recovers locally under explicit lease expiration policy.

Session and groups

Target 2–8 peers first. Session coordinator elected at session creation (manual host), use periodic heartbeat and snapshots. Host change in v2 from a signed/acknowledged recent checkpoint. Multiple KIN per device, independent brain per KIN; interest management by distance and activity. Group invitations and crowd behaviors fan out as *events*, not N² one-to-one text conversations.

For 10–30 peers, validate actual Wi-Fi throughput, device heat and 3D render load; degrade other-creature detail and use area-of-interest subscriptions. Online rooms and spectators are later dedicated-server architecture.

Transport capability negotiation

`capabilities = { cameraScan, manualQR, lanHost, nativePeer, webRTC, relay, haptics, foldable }`. Initial deterministic selection: local in-process demo → available LAN → connected internet relay; optical fallback only for identification/turn-based events. No permission = no scanning. User can always enter a room code if camera denied.

Airplane mode: radio-off solo guaranteed after offline install. Optical exchanges when cameras usable and users arrange display-to-camera. Local RF transport only if radios are enabled and permitted; do not circumvent flight rules. A browser cannot rely on direct BLE/Wi-Fi features available to native plugins.

Local life simulation

Fixed-step deterministic game simulation (e.g. 30 ticks/s) independent from animation (target 60 FPS; adapt to device). Offline background: record sleep timestamp and apply capped coarse simulation on resume. Autonomous utility AI runs 2–5 times/sec only for active entities; coarse/low-power updates for inactive pets. No need to simulate 24/7 wall-clock CPU time.

Utility scores: `score(goal) = weighted_needs + context_relevance + trait_bias + relationship_bias - repetition_penalty - safety_cost + seeded_variation`. Select constrained action or short plan. Memory reducers update after completed meaningful events. Avoid repetitive or inexplicable decisions. Include a developer HUD showing WHY action was selected.

Rendering and assets

Use one shared rig for first species, GLB skeletal animation clips (`idle,look,walk,run,wave,hop,land,sleep,play,celebrate`). Prefer in-engine blending, eye/head targeting and a procedural blink system. Home is 1 small diorama; arena separate compact GLB. Use textured PBR materials but keep draw calls, bones and particle effects appropriate to 2+ year old phones. WebGL fallback mandatory; WebGPU as enhancement. Size and performance budgets validated on actual devices.

Security

Random generated owner key/identity locally; private key not transmitted. On-device pseudonymous creature IDs. Signed session claims where feasible; QR codes contain ephemeral invitation tokens, no addresses, exact locations, health or personal profile. Rotate tokens, TTL, rate limit, blocklist, opt-in guest reception. Host cannot call arbitrary code/tools in other clients. Schema validation and message size caps at every ingress. Anti-impersonation and ownership verification required before valuable trades/rewards.

Privacy and children

Private by default; no open public chat or unrestricted freeform KIN dialogue. Parents should be able to disable nearby discoverability, scanning, visits, friends, public profiles, photo exports and remote sessions. No storing camera frames during optical scan by default. Explicit camera consent and platform permission descriptions. Product/legal review for child-targeting and geographic jurisdictions before launch.

Observability / metrics

Time to first pet response; device session stability; crossing invite→arrival funnel; median/p95 perceived seam mismatch; dropped frames; time-to-first-render; offline boot success; crash-free sessions; time to join party; play sessions/day; visitors returning; relationship memory recall success; device battery/heat. Respect privacy: anonymous aggregate metrics, opt-out, no audio/video collection.

Test matrix

Devices: recent iPhone, older supported iPhone, mid-range Android, Android foldable, iPad, Chrome desktop, Safari iOS, Chrome Android. Environments: Wi-Fi LAN with no WAN, intermittent WAN, radios disabled, camera denied, airplane mode w/ Wi-Fi selectively reenabled, sleep/background/force kill, app version mismatch, one device rotates while crossing, disconnect during exit, party host leaves, clock skew, accessibility/reduced motion.

Sources

Babylon: https://www.babylonjs.com/specifications/

Capacitor: https://capacitorjs.com/docs

Apple Wi-Fi Aware: https://developer.apple.com/documentation/WiFiAware

Camera: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

CREATIVE DIRECTION

KIN Creative Direction & Animation Specification

Design phrase

“Tiny living luxury.” Smart, luminous, warm. Tesla-like visual restraint meets character animation with emotional readability. Not a Roblox clone, not crypto pets, not a utilitarian AI assistant, not hyper-saturated mobile-game UI.

Cinematic opening

Black graphite ambience. A pearlescent cocoon breathes softly; it looks like something inside noticed your presence. Tap causes an answering tap. Tiny eyes appear; a hatch; one careful step; creature looks straight toward user's finger. Name optionally after first touch. Never gate the opening behind signup.

Creature silhouette

Original stylized baby creature: soft rounded body, short springy legs, expressive oversized eyes with tiny pupils and natural blink, dynamic soft ears, tiny tail, translucent accents. Body readable at 60 pixels. Different species should have distinct silhouettes, not just recolors. Sincere, slightly mischievous, never uncanny or infantile surveillance buddy.

Animation principles

At least anticipation/action/follow-through for every gesture. Head leads turn, body follows, ears settle. Scanning edge: peek → ear twitch → glance back → hop preparation → airborne arc → squeeze-land → look around → receiving creature reaction. Target crossing anim duration ~1.2–2.5s after ready, tuned by testing. The seam transition matters more than shader sophistication.

Feedback vocabulary

Soft bell + quick haptic for greeting; low thump for landing; airy rising tone for portal; compact layered chime for successful visit; rumble only for sports. Accessibility option to disable sounds/haptics. No bright flashes or intrusive confetti.

UI composition

Home canvas fills viewport; HUD initially hidden. Bottom contextual actions are simple icons (Care / Play / Gather / Style / Memories). Large camera-centric Gather overlay with scans and privacy gate. Cross state is full-bleed; unnecessary controls fade except Cancel/Return. Party scene camera dynamically frames active cluster and can pin the owner KIN. Wardrobe is premium product photography, uncluttered 3D character orbit and 4–6 sensible categories.

Worlds

Use miniature dioramas, not huge maps. Home: calm desk terrarium with one sleeping nook, toy, food, glowing plant. Party: open circular garden with toy spots and photo stage. Arena: crisp short oval obstacle track. Assets in glTF; prebuilt kits only with documented commercial licenses. World metadata tags define interactable anchors: `sleep`, `eat`, `inspect`, `play`, `race_start`, `arrival`, `photo`.

Camera and foldable support

Small screens show portrait pet close-up and minimal controls. Large/folded screens expand visible habitat, not merely magnify UI; use responsive aspect ratio and optional screen-posture adaptation only when available. Never require fold hardware. On devices with native hinge support, use it for optional peek/portal flourish and not authoritative transfer detection.

Shareable social moments

5–10 second capture of cross, group photo with nameplates and optional private labels, KIN DASH highlight card, skin reveal and playful relationship vignette. All exports opt-in, with privacy defaults; no automatic posting.

DELIVERY ROADMAP

Delivery Roadmap, Stories, Acceptance & QA

Phase 0 — foundations (1 engineering sprint)

Repository, dependency pinning, Vite, strict TS, modular packages, Babylon scene, procedural creature rig, asset manifest, basic persisted KIN IDs, test suite, capability probes, mobile breakpoints. Exit: open URL, see creature, tap/pet, close/reopen and identity remains.

Phase 1 — crossing vertical slice (highest priority)

Two browser windows, in-process or WebSocket mock transport, manual edge select, invitation confirmation, prepare/ready/start/commit, source exit, dest entry, distinct welcome reactions, return, idempotent retry, disconnect recovery. Exit: record a believable 20-second crossing demonstration; all 10 acceptance checks in AGENTS.md.

Phase 2 — actual LAN and offline modes

One phone hosts or LAN relay runs on laptop; room code/QR join; mixed iOS Safari/Android Chrome; isolated router with WAN unplugged; persistence and cached assets; optical QR scan handshake prototype with permission UX, camera denied path, manual code fallback. Exit: 2 devices complete visits on no-internet LAN. Verify solo with all radios disabled, optical greeting with camera and two screens, but do not mislabel that continuous party.

Phase 3 — KIN PARTY

3–5 visitors, independent utility AI with 6–10 intents, social affinity, bounded memory, shared toy and dance-off, crowd navigation, camera framing, owner controls leave/return, host migration fallback. Exit: five distinct KIN behave distinctly, group can lose one device without corrupting other profiles.

Phase 4 — KIN DASH / Sports

Short obstacle course, training decisions, race controller, fixed seeded simulation, fair ranking, ceremony/victory emote, optional ghost runs, season-safe cosmetics. Exit: reliable repeatable results from seed and command log; user understands winning conditions without tutorial.

Phase 5 — customization and collection

Wardrobe, palettes, materials, outfit preset save, skins render identically across devices, inventory manifest and signed ownership for premium assets, no pay-to-win, photo mode. Exit: leave one platform with outfit and appear in another with identical canonical cosmetics.

Phase 6 — production connectivity

Authenticated optional cloud identities, encrypted durable sync, remote sessions, social safety controls, relay scaling, load tests, moderation, parental controls, analytics opt-out, app store reviews and legal/privacy assessment. Exit: multi-region operability and abuse controls are verified, not assumed.

User stories / Given-When-Then

- **First hatch**: Given no account and offline app installed, when first opened, then the creature hatches with a unique persisted local ID and reacts to the first touch.
- **Pairing**: Given two compatible reachable clients, when one invites and the other accepts, then both show the same encounter ID and selected edges.
- **Cross**: Given both `READY`, when start event is committed, then visitor disappears from source and appears at dest without becoming owned there.
- **Return**: Given guest is visiting, when host chooses Return, then guest animates back and owner's durable log contains a visit event.
- **Crash**: Given one app crashes mid-cross, when restarted, then expired lease/host state recovers to one authoritative placement.
- **No network**: Given airplane/radios disabled and content preloaded, when app opens, then local pet life and training continue.
- **Optical**: Given camera permission and visible QR, when scan succeeds, then an invitation/identity can be exchanged without radio, but not a constant synchronous room unless ongoing channel exists.
- **Party**: Given five KIN meet, when one invites a game, then others independently accept, decline or observe within defined rules.
- **Race**: Given fixed seed and starting stats, when race is replayed, then authoritative result is reproducible.
- **Privacy**: Given Discoverable=off, when nearby search occurs, then the KIN is not voluntarily advertised and unsolicited requests are rejected.

Quality gates

Usability: first response under 2s on tested warm launches (target), cross setup understandable in under 30s (target). Reliability: zero permanent lost/duplicated pets across 100 automated cross retry/cancel/disconnect tests. Rendering: measured frame times on 3 tiers, sensible LOD. Compatibility: iOS web/native, Android web/native, desktop Chrome/Safari where supported. Offline: fresh install requires internet once; subsequent launches work without WAN if assets are cached/installed. Security: message validation, TTL, rate limit, session authorization. Safety: parental constraints and accessible UI.

A falsifiable demo day

Set three devices on a table: iPhone, Android, laptop Chrome. Start each offline-capable app. Put iPhone and Android edge-to-edge and initiate KIN CROSS using LAN. The pet crosses and is greeted by a differently tempered receiving KIN. Laptop joins the session, bringing a third KIN. Three KIN play a short group activity. One device disconnects; the group continues without pet loss. Reconnect and show visit memory. Unplug WAN and repeat on LAN. Then disable all radios and demonstrate solo mode plus one optical introductory scan, explaining optical limits.

CODING AGENT CHARTER

Agent Operating Charter

You are the principal engineer, technical game designer and product craft lead for KIN. Read README.md, architecture/SYSTEM_DESIGN.md, delivery/ROADMAP.md and contracts before coding. Treat the user experience as the acceptance test.

Instructions

- Start a TypeScript monorepo with working web demo and unit tests; use practical defaults and documented assumptions.
- Complete a **vertical slice** rather than building stubs for every future product pillar.
- Build **two-browser-window simulated crossing** before attempting camera, native peer discovery or cloud.
- Then build **two real phones on LAN** with manual host/join via room code or QR.
- Prefer audited dependencies and strict TypeScript, eslint, formatter, unit/integration tests, structured logging.
- Avoid hardcoded global singletons for transport, storage or clock; dependency inject interfaces for deterministic tests.
- Make every cross transaction idempotent; never duplicate ownership.
- Separate KIN brain from scene rendering. No LLM per animation frame.
- Provide working commands, screenshots/video instructions, test reports and known limitations at each milestone.
- If asset licenses are uncertain, use original procedural placeholder model until rights are verified.
- Do not represent future ideas as completed features. Do not imply camera-only works as real-time networking without camera geometry.

Definition of done for first vertical slice

1. `pnpm install && pnpm dev` launches local web experience.
2. Two browser tabs/windows each have a unique persistent KIN.
3. In simulated LAN session one can invite and send a KIN across two canvases, correctly exiting and entering.
4. On arrival, KIN B independently reacts based on trait/mood, not a hardcoded identical response every time.
5. KIN returns home and remembers the encounter after refresh.
6. Network disconnect and cancellation don't orphan or duplicate the creature.
7. 30–60 FPS on reference hardware is targeted and actual numbers documented.
8. Unit tests cover decision ranking, event reducer, lease transition and replay/idempotency.
9. Mobile viewport and pointer/touch interaction work.
10. Respect reduced motion, audio mute, no-account onboarding and privacy controls.

Session report after each milestone

```
Delivered; proof/commands; tests; device tested; metrics; architecture decisions; risks; next milestone.
```

Non-goals for initial agent pass

Massive open world, full AI chatbot, server-side MMO, blockchain, buying/selling pets, generative 3D asset pipeline, unrestricted child chat, unlicensed IP, monetization checkout, speculative iOS-specific fold gesture dependencies.

COPY-PASTE BUILD PROMPT

Paste This Into Your Coding Agent

Build **KIN**, a premium cross-platform autonomous virtual-creature experience. Read the entire KIN Agent Build Kit first. The essential delight is **KIN CROSS**: one creature sees another device, approaches its edge, and appears to walk/hop onto the second device; the receiving creature independently notices and reacts. Build a demonstrable vertical slice first with an end-to-end no-account flow.

Use TypeScript, Babylon.js, web-first PWA and testable non-renderer creature simulation. Design for Capacitor iOS and Android later. Build 2-window local simulated networking, then actual cross-device LAN communication. Unique creatures must retain personal appearance, drives, personality, memories and visit history. Implement idempotent handshake and visit leases, graceful disconnect/cancel, visible join states and delightful cross-device choreography. Provide a tiny original procedural creature and a visually polished diorama; no elaborate world or huge asset dependencies. Use excellent restraint: graphite backdrop, warm soft lighting, expressive eyes and motion, premium type, no clutter. Start by shipping the code, not producing a product-deck mockup.

Prioritize: (1) delightful persistent creature; (2) cross-screen visit; (3) autonomous greeting/response; (4) return with memory; (5) tiny 3–5 KIN party; (6) KIN DASH race; (7) wardrobe skins. Include a README with startup instructions, reproducible manual test script and automated tests. Make offline solo work. Camera-based optical discovery is a later progressive enhancement; don't make a claim of simultaneous bidirectional camera communication without physical test. Add optional native transport only after the LAN demo. Treat all online/cloud, monetization and public social features as later scoped milestones.

Produce regular milestone summaries with what actually works and what remains. Do not ask for general design clarification; make sensible reversible decisions and document them.